

# Data Structure

## Level 1

### Singly Linked List Data Structures Operations

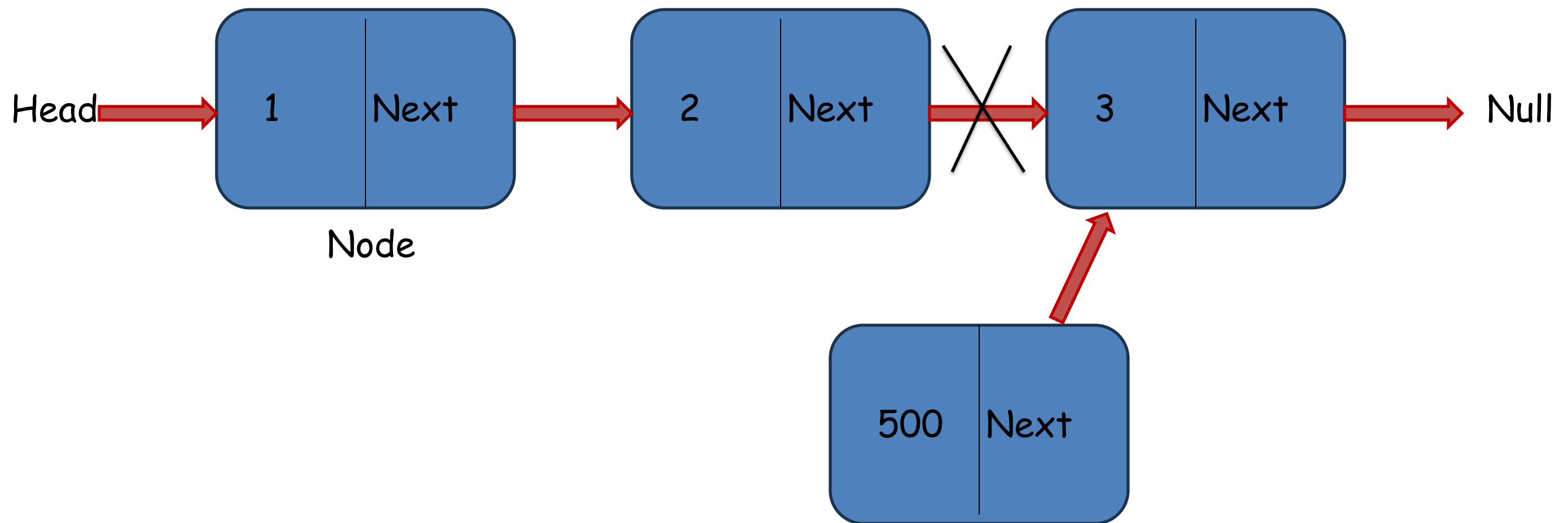


# Data Structure

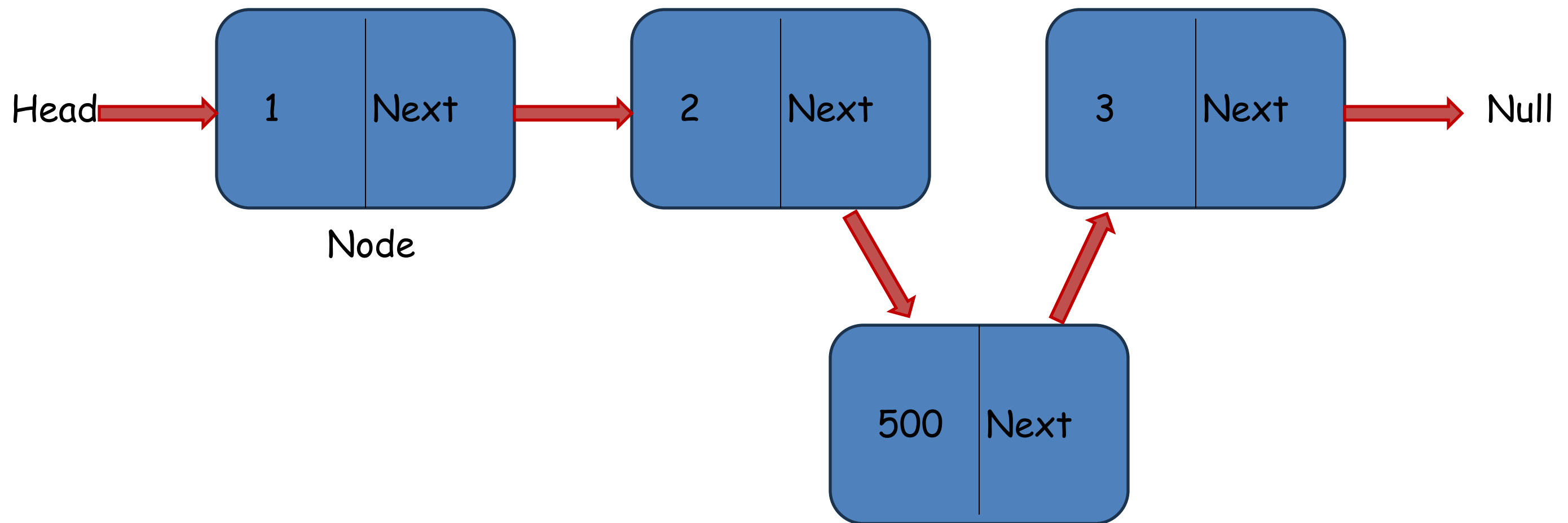
Level 1

## Insert After

# Linked List Data Structure



# Linked List Data Structure





# Definition

InsertAfter() Function:

Inserts a new node after a given node in the linked list.

■ Requirements:

■ A valid pointer to the previous node

■ A value to insert



## Explanation

- ◆ First, we create a linked list using `InsertAtBeginning()`.
- ◆ Then we search for a specific node using `Find()`.
- ◆ After finding the node, we call `InsertAfter()` to insert a new node immediately after it.
- ◆ Pointer connections are updated carefully to preserve the list structure.

## Procedures (Insert After)

- 1 Create an empty linked list
- 2 Insert elements at the beginning
- 3 Print the list
- 4 Call Find(head, value) to locate target node
- 5 Pass returned pointer to InsertAfter()
- 6 Create new node
- 7 Set new\_node->next = prev\_node->next
- 8 Set prev\_node->next = new\_node
- 9 Print updated list



## Example

After inserting values (1,2,3,4,5) at beginning:

List becomes:

5 → 4 → 3 → 2 → 1 → NULL

Now inserting **500** after 2

→ Updated list:

5 → 4 → 3 → 2 → 500 → 1 → NULL



## Note

The order of pointer updates is very important:

- First connect new node to next node
- Then connect previous node to new node



# Comparison

- Insert at Beginning
  - ✓  $O(1)$  time
  - ✓ No search required
  
- Insert After Specific Node
  - ✓ Requires search first
  - ✓ Overall time complexity:  $O(n)$
  - ✓ More pointer operations

## **Warning**

If `prev_node` is `NULL`, insertion is not possible and may cause a crash.  
The program correctly checks this condition.

## ✨ Summary

- ✓ InsertAfter() adds a node after a specific node
- ✓ Requires finding the node first
- ✓ Pointer updates must be done carefully
- ✓ Time complexity is  $O(n)$  due to search
- ✓ Maintains linked list structure properly

Thank you 😊